

EXPERIMENT - 05

Aim

Hands on Solidity Programming Assignments for creating Smart Contracts

Theory

Solidity is a high-level, statically typed programming language designed for writing **smart contracts** that execute on blockchain networks. A smart contract contains predefined rules and functions that are executed when specific conditions are met. In this experiment, Solidity programs are written, compiled, deployed, and tested using the **Remix IDE** and **Remix VM**.

1. Primitive Data Types and Variables

Solidity provides several primitive data types such as `uint` for unsigned integers, `int` for signed integers, `bool` for Boolean values, `address` for Ethereum addresses, and `string` or `bytes` for textual and binary data. Variables can be classified as **state variables**, **local variables**, and **global variables**. State variables are permanently stored on the blockchain, while local variables exist only during function execution. Global variables provide blockchain-related information such as `msg.sender`, `msg.value`, and `block.timestamp`.

2. Functions

Functions are blocks of code used to perform specific operations in a smart contract. They can accept input parameters and return one or more values. Solidity supports **pure** and **view** functions. A `pure` function does not read or modify blockchain state, whereas a `view` function can read state variables but cannot modify them.

3. Function Visibility

Visibility determines how functions and variables can be accessed. Solidity provides four visibility levels:

- **Public:** Can be accessed both internally and externally.
- **Private:** Can be accessed only within the contract where it is declared.
- **Internal:** Can be accessed within the contract and derived contracts.
- **External:** Can be called from outside the contract.

Visibility helps control access to contract functionality and data.

4. Modifiers

Modifiers are reusable blocks of code that can change or restrict the execution of a function. They are commonly used for **access control**. For example, an `onlyOwner` modifier can ensure that only the account that deployed the contract can execute a particular function.

5. Constructors

A constructor is a special function that is executed **only once**, when a smart contract is deployed. It is generally used to initialize state variables or store information about the account deploying the contract.

6. Control Flow Statements

Solidity supports conditional statements such as `if-else`, which allow the contract to make decisions based on conditions. It also supports loops such as `for`, `while`, and `do-while` for repeated execution. Loops must be used carefully because excessive execution can increase gas consumption and may cause transactions to fail.

7. Arrays

Arrays are ordered collections of elements of the same type. They can be **fixed-size** or **dynamic-size**. Dynamic arrays can grow during contract execution using functions such as `push()`. Arrays are useful for storing collections of related values.

8. Mappings

Mappings are key-value data structures used for efficient data retrieval. For example, `mapping(address => uint)` can associate an Ethereum address with a numerical value such as a balance. Mappings provide efficient lookups but do not support direct native iteration.

9. Structures

A **struct** is a user-defined data type that groups multiple related variables together. For example, a `Student` structure can contain a student's name, age, and roll number. Structures make it easier to organize complex information within a smart contract.

10. Enumerations

An **enum** is a user-defined data type consisting of a fixed set of named values. For example, a contract can define a status as `Pending`, `Approved`, or `Rejected`. Enums improve readability and make the state of a contract easier to understand.

11. Data Locations

Solidity uses different data locations depending on how data needs to be stored:

- **Storage:** Permanent location used for state variables and stored on the blockchain.
- **Memory:** Temporary location that exists only while a function is executing.
- **Calldata:** Non-modifiable and non-persistent data location generally used for parameters of external functions.

Choosing the appropriate data location is important for efficient smart contract execution.

12. Ether and Wei

Ether (ETH) is the native cryptocurrency of the Ethereum blockchain. Internally, Ethereum uses **Wei**, which is the smallest unit of Ether.

1 Ether = 10¹⁸ Wei

Using Wei allows Ethereum transactions to represent very small amounts of Ether accurately.

13. Gas and Gas Price

Every operation performed on the blockchain requires computational resources. **Gas** represents the amount of computational work required to execute a transaction or smart contract operation. The **gas price** determines the amount of Ether paid per unit of gas. Therefore, more computationally intensive operations generally require more gas.

14. Sending Transactions

Transactions are used to interact with smart contracts and modify their blockchain state. Functions that change state generally require a transaction to be submitted and executed by the blockchain. Ether can also be transferred using mechanisms such as `transfer()`, `send()`, or `call()`.

15. Remix IDE and Remix VM

Remix IDE provides an environment for writing, compiling, deploying, and testing Solidity smart contracts. The **Remix VM** provides a simulated blockchain environment where contracts can be deployed and tested without using real Ether. This makes it suitable for learning and practical experimentation.

In this experiment, the different Solidity concepts are implemented in a smart contract, compiled using the Solidity compiler, deployed on Remix VM, and tested through the available contract functions. This provides practical understanding of how Solidity programs operate as blockchain-based smart contracts.

Code

```
1 pragma solidity ^0.8.20;
2 contract Experiment5 {
3     uint public number = 100;
4     int public signedNumber = -50;
5     bool public flag = true;
6     address public owner;
7     string public name = "Rutuja";
8     constructor() {
9         owner = msg.sender;
10    function add(uint a, uint b)
11        public
12        pure
13        returns (uint)
14    {
15        return a + b;
16    }
17    function getNumber()
18        public
19        view
20        returns (uint)
21    {
22        return number;
23    }
24    modifier onlyOwner() {
25        require(msg.sender == owner, "Not the owner");
26        _;
27    }
28    function changeNumber(uint newNumber)
29        public
30        onlyOwner
```

```
31    {
32        number = newNumber;
33    }
34    function checkNumber(uint num)
35        public
36        pure
37        returns (string memory)
38    {
39        if (num > 50) {
40            return "Number is greater than 50";
41        } else {
42            return "Number is 50 or less";
43        }
44    }
45    function sumNumbers(uint n)
46        public
47        pure
48        returns (uint)
49    {
50        uint sum = 0;
51
52        for (uint i = 1; i <= n; i++) {
53            sum += i;
54        }
55
56        return sum;
57    }
58    uint[] public numbers;
59    function addNumber(uint num)
60        public
61    {
```

default_workspace

Compile Home Settings experiment.sol 1 x

```
62     numbers.push(num);
63 }
64 mapping(address => uint) public balances;
65 function setBalance(uint amount)
66     public
67 {
68     balances[msg.sender] = amount;
69 }
70 struct Student {
71     string name;
72     uint age;
73     uint rollNo;
74 }
75 Student public student;
76 function setStudent(
77     string memory _name,
78     uint _age,
79     uint _rollNo
80 )
81     public
82 {
83     student = Student(_name, _age, _rollNo);
84 }
85 enum Status {
86     Pending,
87     Approved,
88     Rejected
89 }
90 Status public status;
91 function setStatus(Status _status)
92     public
```

```
93 {
94     status = _status;
95 }
96 string public storedName;
97 function setName(string memory _name)
98     public
99 {
100     storedName = _name;
101 }
102 function displayName(string calldata _name)
103     external
104     pure
105     returns (string memory)
106 {
107     return _name;
108 }
109 receive() external payable {}
110 function getBalance()
111     public
112     view
113     returns (uint)
114 {
115     return address(this).balance;
116 }
117 }
```

Implementation

●

number

Call

0:uint256: 150

●

changeNumber (uint256)

20

Transact

Calldata Params

Auto-Fill with AI Samples

●

number

Call

0:uint256: 20

Functions

owner

●

owner

Call

0:address: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

●

add (uint256, uint256)

Call

100

200

Calldata Params

Auto-Fill with AI Samples

0:uint256: 300

● **balances** (address) Call

0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

Calldata  Params 

 Auto-Fill with AI Samples

0:uint256: 0

Functions

balance

● **setBalance** (uint256)

50

Transact

Calldata  Params 

 Auto-Fill with AI Samples

● **balances** (address) Call

0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

Calldata  Params 

 Auto-Fill with AI Samples

0:uint256: 50

● **setStudent** (string, uint256, uint256)

rutuja

21

03

Calldata  Params 

 Auto-Fill with AI Samples

Transact

● **student** Call

0:string: name rutuja

1:uint256: age 21

2:uint256: rollNo 3

Conclusion

Thus, the fundamentals of **Solidity programming and smart contract development** were successfully implemented using Remix IDE. The experiment provided practical understanding of variables, functions, modifiers, constructors, control-flow statements, data structures, data locations, and blockchain transactions. The smart contract was successfully **compiled, deployed, and executed on Remix VM**, strengthening the understanding of Ethereum smart contract development.